MLOps Scoping – SocialNexus

Team Members:

1. Atul Kumar
2. Sarthak Kagliwal
3. Gargi Kulshreshtha
4. Harshita Prabhu
5. Siddhant Singh

1. Introduction

This project aims to create an end-to-end pipeline that processes user queries, retrieves relevant tweets using Elasticsearch, and a ranking algorithm, and generates insights using a Retrieval-Augmented Generation (RAG) model. The system leverages Twitter data as the source and OpenAI APIs for insights generation, aiming to provide meaningful analysis for user-defined queries.

2. Dataset Information

1. **Dataset Introduction**: We will be summarizing top 1000 tweets based on the query asked by the user. In this dataset we have multiple csv files in which we have data of people like there tweets, followers, following, etc. with the help of these details we'll be using our model to fetch out the relevant tweets from it.

2. Data Card:
   User Information: Includes userid, username, acctdesc, location, following, followers, totaltweets, and usercreatedts.

   Tweet Details: Includes tweetid, in_reply_to_status_id, is_quote_status, quoted_status_id, quoted_status_userid, and extractedts,original_tweet_userid, original_tweet_username, and other tweet-related identifiers.

3. **DataSources:**
   https://www.kaggle.com/datasets/bwandowando/ukraine-russian-crisis-twitter-dataset-1-2-m-rows/versions/508?select=0831_UkraineCombinedTweetsDeduped.csv.gzip .

4. **Data Rights and Privacy**:
   - Publicly Available: We are downloading the datasets from Kaggle which is available for public online.
   - GDPR: Potential user metadata within tweets. We must ensure any personal info or user IDs are handled per platform regulations (pseudonymization or partial redaction if necessary).

3. Data Planning and Splits:

- For the data preprocessing steps, we will be ensuring date columns (usercreatedts, extractedts) are parsed as datetime objects for easier manipulation.
- Also, we will be handling the missing values. In addition to this we will have to deal with the duplicate values. For instance, removing the duplicate tweets from the same user.
- Normalize text fields (username, location) for consistency.
- Convert categorical fields (e.g., is_quote_status) into boolean or categorical types.
- Also, we will be dealing with the outliers' value. Ex: In totaltweets column, the range is from 1 to 4M.
- For splitting, we will use the ranking algorithm. Engagement metrics would be followers, following, no of retweets, no of reply, Text characters, total tweets. We will be assigning the weights to prioritize factor
- Then we will sort the tweets on the calculated relevance score.

4.GitHub:

https://github.com/Atulya1/MLOps

READMEfile:
https://github.com/Atulya1/MLOps/commit/5c6eb67642fa60e2b54a56348df11717dcb33239

5. Project Scope

5.1 Problems:

1. High-Volume Data: Efficiently ingesting and indexing large sets of tweets.
2. Relevant Retrieval: Need both keyword filtering (Elasticsearch) and semantic search (Vector DB) for accurate results.
3. Meaningful Insights: Generating human-like summaries/Q&A from raw tweets using an LLM.

5.2 Current Solutions

1. Traditional social listening tools primarily rely on keyword matching.
2. Some solutions use advanced ML, but they may be costly or not customized for specific user queries.

5.3 Proposed Solutions

1. Azure-Based Ingestion: Blob Storage + Azure Functions to handle data at scale.
2. Data Preprocessing: Clean, rank, and store only top tweets in Elasticsearch.
3. Vector Database: Store embeddings for semantic relevance.
4. RAG Model: Use an Azure Function that leverages an LLM to produce real-time insights from retrieved tweets.

6. Current Approach Flow Chart and Bottleneck Detection

Flow Diagram Explanation

1. CSV Files → Data Lake (Blob Storage):

   Azure Functions retrieve batches (1,000 tweets).

2. Data Preprocessing:

   Perform data cleaning and ranking, preparing tweets for indexing.

3. Elasticsearch Processing:

   Index the top tweets and query them to get the top 50 for a given user question.

4. Vector Search (Embeddings):

   - User query is embedded; relevant tweets also have embeddings stored.
   - Perform semantic similarity search to refine the top results.

5. RAG Model (Azure Function):

   - Combines the final set of relevant tweets as context for the large language model.
   - Generates insights/answers returned to the Backend Service.

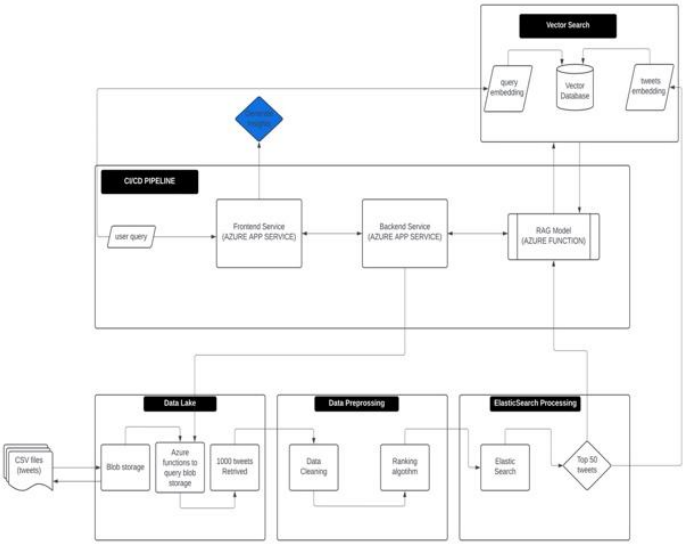
6. Azure App Services (Frontend & Backend):

   - The user query flows from the frontend to the backend, which orchestrates calls to Elasticsearch, the Vector DB, and the RAG model.
   - CI/CD ensures consistent deployments across all Azure services.
   - Elasticsearch Indexing: If we process millions of tweets at once, we need to manage ES cluster size and indexing rate.
   - Embedding Generation: Generating embeddings for many tweets can be computationally expensive; consider pre-computing in batches.
   - Azure Function Timeouts: If RAG function calls are too large or concurrency is high, we may need a premium plan or micro-batching.

7.Improvements:

- Cache embedding results to avoid repeated computations.
- Scale Elasticsearch or partition the data for high throughput.
- Use event-driven Azure Functions for more granular data ingestion.

Flowchart:



7. Metrics, Objectives, and Business Goals

1. Key Metrics:

- Indexing Throughput: tweets per minute successfully stored in Elasticsearch.
- Query Latency: from user query to final insights.
- Semantic Retrieval Accuracy: how relevant the top results are (precision/recall).
- LLM Response Quality: user satisfaction or a manual evaluation metric.

2. Objectives:

- Enable near-real-time ingestion and retrieval for Twitter-scale data.
- Provide quick summarization or Q&A from tweets.
- Demonstrate a fully scalable, Azure-deployed architecture.

3. Business Goals:

- Offer a customizable social-listening solution for enterprise clients.
- Use advanced AI to mine actionable insights from Twitter (or similar) data.

8. Failure Analysis

- Data Lake Inconsistencies: CSV files missing columns or corrupt data.
- Mitigation: Validate schema on ingestion; reject/fix malformed files.
- Elasticsearch Overload: Large volumes or big queries might degrade performance.
- Mitigation: Scale out cluster, optimize queries, tune shards.
- Azure Function Timeout: If embedding generation or RAG inference is too slow.
- Mitigation: Use premium function plans, break tasks into smaller chunks.
- LLM Costs/Rate Limits: Repeated calls to an LLM can get expensive or hit limits.
- Mitigation: Cache frequent queries, or batch requests to reduce overhead.

9. Deployment Infrastructure

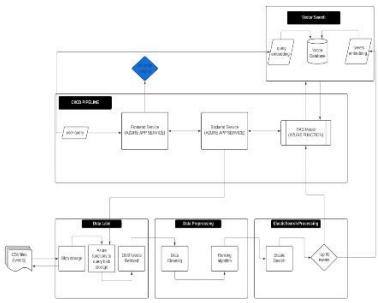
1. Infrastructure:

- Blob Storage: For raw CSV files.
- Azure Functions: For data ingestion, transformation, RAG inference.
- Azure App Services: Frontend (UI) + Backend (APIs).
- Elasticsearch: For keyword-based and advanced text searching.
- Vector Database: Could be an external service (e.g., Pinecone) or use Elasticsearch's k-NN features.
- CI/CD Pipeline: e.g., Azure DevOps or GitHub Actions to deploy code to Functions, App Services, and manage infra as code (ARM/Bicep/Terraform).

2. Supported Platforms:

- Primarily Azure, but the same logic can run on other clouds if needed (with corresponding services).

3. Flowchart:



3.a. Azure
3.b Github Actions
3.c Hugging Face

10. Monitoring Plan

- Metrics to Monitor:

1. Azure Functions: Execution times, failures, concurrency.

2. Elasticsearch: Cluster health, node performance, query latency.

3. Vector DB: Embedding insert/search latency.

4. App Services: Response time, HTTP error rates.

- Why:

1) Ensures the pipeline handles spikes in data volume and user queries.
2) Identifies bottlenecks (e.g., slow ingestion or queries).

- Tooling: Azure Monitor, Application Insights, Kibana for Elasticsearch logs, etc.

11. Success and Acceptance Criteria

- Functional:

1. Successfully ingest CSV tweets to Blob and process them into Elasticsearch with minimal errors.

2. Return relevant tweets in under 2 seconds on average for user queries.

3. The RAG model provides coherent, context-based responses to at least 80% of test queries.

- Non-Functional:

1. Pipeline maintains an uptime of 99%+ across all Azure services over a 30-day window.

2. Clear logs and monitoring dashboards are in place for debugging and performance optimization.

12. Timeline Planning

Milestone	Target Date	Deliverables
Data Lake & Blob Setup	Week 1 (28 Jan-4 Feb)	Blob containers, ingestion function
Data Preprocessing Dev	Week 2 (5th Feb-12th Feb)	Cleaning & ranking scripts, tests
Elasticsearch Integration	Week 3 (13th Feb-20th Feb)	ES index creation, data ingestion
RAG Model (Azure Function)	Week 4 (21st Feb-28th Feb)	Embedding + inference logic, LLM calls
Frontend & Backend Services	Week 5 (1st March-15th March)	Basic UI, API routes for queries
CI/CD Pipeline Completion	Week 6 (16th March-30rd March)	Automated deployments to Azure
Final Testing & Documentation	Week 7 (31st March- 10th April)	Load tests, final docs, handoff

13. Additional Information

- Scalability Options: Potential to integrate event-driven ingestion for real-time updates.
- License Compliance: Must verify the original source of CSV tweets allows storage and analysis at scale.
- Future Enhancements: Multi-lingual tweet support, advanced NLP tasks like sentiment analysis, and integration with a knowledge graph.